

ClaimsFlow

System Analysis Report

Strengths · Gaps · Strategic Improvements

CIC Insurance Group PLC

Prepared: May 10, 2026

Sector: InsurTech / Medical Claims Automation

1. Executive Summary

ClaimsFlow is a well-structured, full-stack medical insurance claims management platform built for CIC Insurance Group PLC. The system covers the complete claim lifecycle — from batch submission and multi-model OCR extraction through maker-checker dual approval, fraud signal detection, provider management, and reporting. The technology stack (NestJS + React 18 + PostgreSQL + BullMQ) is modern, appropriate for the problem domain, and properly containerised with Docker.

Key finding: The platform is approximately 85% feature-complete and represents a strong foundation. However, zero automated test coverage, several disabled integrations, and the absence of production-grade hardening create meaningful risk before a full production rollout. The recommendations in this report are prioritised by impact and sector-specific relevance.

2. What the System Does Well

✓ Maker-Checker Dual Approval

The four-eyes principle is correctly implemented as a first-class workflow. Every claim passes through an independent maker review then a checker review before finalisation — a regulatory requirement in Kenyan insurance and an industry best practice globally.

✓ Multi-Model OCR Pipeline

The system intelligently routes documents through Gemini Vision (cloud), Tesseract.js (client-side), and OLLAMA (local LLM) depending on availability. Graceful fallback prevents total failure when a provider is down.

✓ Fraud Signal Detection

Seven automated fraud patterns are checked at submission time: duplicate invoice numbers, provider mismatch, negative amounts, unusually high claim values, and more. This gives the fraud queue officers a pre-filtered, actionable queue.

✓ Activity Audit Trail

A global NestJS interceptor logs every user action with IP address, user ID, and timestamp. This is essential for both regulatory audit submissions and internal incident investigation.

✓ Barcode-Based Claim Identity

Code-128 barcodes embedded in PDFs give each claim a scannable physical identity (CIC-YYYYMMDD-XXX-XXXXXX). Physical and digital records stay in sync even when documents are printed, scanned, and re-ingested.

✓ Role-Based Access Control

Five distinct roles (admin, supervisor, claims officer, checker, fraud officer) with fine-grained resource-action permissions ensure that each user sees only what they are authorised for — preventing accidental or deliberate data leakage across departments.

✓ Modular Architecture

19 NestJS modules cleanly separate concerns. New integrations (payment gateway, e-health systems) can be added as new modules without touching existing logic.

3. Gaps & What I Would Do Differently

3.1 Automated Testing — Critical Gap

The single largest risk in this codebase is the **complete absence of automated tests**. With ~49,000 lines of TypeScript and 179+ API endpoints, any refactoring, dependency upgrade, or new feature can silently break existing workflows. In an insurance context where incorrect claim decisions have financial and legal consequences, this is not acceptable for production.

Recommended actions:

- Write Jest unit tests for all service classes (OCR routing logic, fraud signal detection, assignment strategies, completeness validation rules).
- Write NestJS integration tests using SuperTest against a test Postgres database — do not mock the database; use real queries to catch schema migrations.
- Write Playwright end-to-end tests for the three most critical user journeys: batch upload → OCR → maker approve → checker approve, provider onboarding, and fraud queue triage.
- Set a coverage gate of $\geq 80\%$ for the service layer in CI — PRs below threshold are blocked from merging.

3.2 EDMS & eOxygen Integrations — Skeleton Only

The Electronic Document Management System (EDMS) and eOxygen payment system integrations are stub files with no real API calls. Without EDMS, approved claims exist only inside ClaimsFlow and are not archived to the enterprise document store. Without eOxygen, payment disbursement cannot be triggered from within the platform — a manual step remains.

Recommended actions:

- Obtain EDMS API specifications from the enterprise IT team and implement authenticated document push on claim approval.
- Obtain eOxygen API specifications and implement automated payment initiation with a reconciliation webhook listener to update claim status to 'paid'.
- Add integration-level circuit breakers (e.g., using the 'cockatiel' library) so that EDMS/eOxygen downtime degrades gracefully instead of failing claims.

3.3 Two-Factor Authentication — Backend Disabled

The 2FA backend controller and service files are renamed with a **.disabled** extension, meaning the frontend 2FA setup page is non-functional. Given that ClaimsFlow handles sensitive personal health and financial data, 2FA should be mandatory for all privileged roles (admin, supervisor, checker) per data protection best practice.

Recommended actions:

- Re-enable the TOTP controller and service, wire them back into app.module.ts.
- Make 2FA mandatory (not optional) for the admin, supervisor, and checker roles — enforced server-side in the JWT strategy.
- Add SMS-based OTP as a fallback via Africa's Talking (already integrated) for users without a TOTP app.

3.4 Security Hardening

Several security settings are acceptable for MVP but need tightening before handling real policyholder data:

Issue	Current State	Recommended Change
JWT Token Lifetime	7 days — very long for session tokens	Shorten to 15 minutes + introduce refresh tokens (30-day sliding window)
CORS	All localhost ports allowed — insecure	Restrict CORS to the specific production domain only
Rate Limiting	Global throttler only (no per-endpoint limits)	Add stricter limits on /auth/login (5 req/min), /auth/reset-password, and batch-upload endpoints
CSRF Protection	No CSRF middleware configured	Add csurf or implement SameSite=Strict cookies for session-bearing routes
Docker Secrets	Hardcoded credentials in docker-compose.yml	Use Docker secrets or an environment variable file excluded from version control

3.5 Database Performance & Reliability

The Prisma schema has 30 models but sparse indexing beyond primary keys. At production claim volumes (100,000+ claims per year is typical for a mid-size insurer), queries on claim status, provider ID, and submission date will perform full table scans.

Recommended actions:

- Add composite indexes on (providerId, status), (batchId, status), and (submittedAt DESC) in schema.prisma.
- Implement soft-deletes using a deletedAt nullable timestamp — hard deletes violate insurance record retention regulations.
- Add database-level constraints for enum fields (Prisma enums are not enforced at the database layer by default).
- Consider read replicas for the reports module — heavy analytics queries should not compete with transactional claim processing.

3.6 API Documentation — Missing

179+ REST endpoints have no Swagger/OpenAPI documentation. Provider integrations, third-party system connections, and internal developer onboarding all depend on this.

Recommended actions:

- Add @nestjs/swagger and decorate all controllers and DTOs — NestJS generates the full spec automatically.
- Enable the Swagger UI at /api/docs behind an admin-only bearer token guard.
- Publish the OpenAPI JSON to a developer portal or Postman collection for external provider integrations.

3.7 Observability & Incident Response — Absent

Currently, logging goes to local rotating files only. In production, an undetected queue backlog or OCR failure can silently block claims for hours before anyone notices.

Recommended actions:

- Ship Winston logs to a centralised aggregator — AWS CloudWatch, Datadog, or an on-premise ELK stack (Elasticsearch + Logstash + Kibana).
- Add structured JSON log format with correlation IDs per request so a single claim's lifecycle can be traced across OCR, workflow, and notification logs.
- Create a BullMQ dashboard (BullBoard) for real-time queue monitoring — visible to supervisors, not just engineers.
- Set up alerts for: queue depth > 500, OCR failure rate > 5%, notification delivery failures.

4.1 Real-Time Collaboration on Claims

Currently, two reviewers can inadvertently open the same claim simultaneously, leading to conflicting decisions. Insurance workflows benefit enormously from real-time awareness — who else is viewing a claim, live status changes, and instant notifications when a colleague completes their step.

Implementation:

- Add a WebSocket module using @nestjs/websockets + Socket.io (or native WebSocket).
- Emit events on: claim status change, new comment, assignment, approval/rejection.
- Show a 'John is reviewing this claim' presence indicator on the claim detail page.
- Optimistic locking: lock a claim for editing while a reviewer is actively working on it (release after 5 minutes of inactivity or explicit release).

4.2 SLA Tracking & Escalation Engine

The Insurance Regulatory Authority of Kenya (IRA) requires insurers to settle or respond to claims within defined timeframes. ClaimsFlow has no SLA tracking — a critical compliance gap. Claims approaching their regulatory deadline should be automatically escalated, flagged, and reported.

Implementation:

- Add SLA configuration per claim type (e.g., inpatient 72 hours, outpatient 24 hours, pre-authorisation 4 hours).
- Add a scheduled BullMQ job (cron) that runs every 15 minutes to identify claims approaching SLA breach (>80% elapsed).
- Auto-escalate breaching claims: reassign to supervisor, send SMS to responsible officer, and flag with a red SLA badge in the UI.
- Add an SLA compliance report to the Reports module — percentage within SLA by claim type, officer, and month.

4.3 Provider Self-Service Portal

Hospitals and clinics currently have no direct window into their submitted claims. They rely on phone calls and emails to CIC staff for status updates — creating unnecessary inbound call volume and delays. A read-only (or limited-action) provider portal would dramatically reduce operational overhead.

Implementation:

- Add a Provider role with read-only access limited to their own claims (access control already partially implemented via providerAccessGuard).
- Build a ProviderPortal page showing: submitted claims, status breakdown, approval rates, outstanding documents required.
- Enable providers to upload supplementary documents directly to an existing claim (with a notification to the assigned officer).
- Add SMS/email status update opt-in so providers get push notifications without logging in.

4.4 AI-Powered Claim Pre-Population

The OCR pipeline extracts text from submitted documents but does not yet auto-fill claim form fields. Officers manually re-type data that OCR has already extracted — wasted effort and a source of transcription errors. The infrastructure (Gemini Vision, Tesseract, document classification) is already in place.

Implementation:

- Extend the OCR service to return structured JSON (patient name, diagnosis codes, invoice amounts, dates, provider number) in addition to raw text.

- Map extracted fields to claim DTO fields and auto-populate the claim form in the UI — officer reviews and confirms rather than types from scratch.
- Show confidence scores alongside each auto-filled field so reviewers know which fields need verification.
- Feed officer corrections back as labelled training data to improve extraction accuracy over time (active learning loop).

4.5 Comprehensive Fraud Score (ML Model)

The current fraud detection checks 7 rule-based patterns. This is a solid starting point, but sophisticated fraud — split billing, upcoding, collusion between providers and policyholders — is not detectable by simple rules. A machine learning fraud score would surface non-obvious anomalies.

Implementation:

- Train a gradient-boosted tree (XGBoost or LightGBM) on historical approved/rejected claims with fraud labels as a starting point.
- Features: provider claim frequency, average invoice amount vs. provider historical mean, claim volume spikes, ICD-10 code clustering per provider.
- Expose the model as a lightweight Python FastAPI microservice — ClaimsFlow backend calls it as an async job during batch processing.
- Display fraud probability score (0–100) alongside existing signal flags in the fraud queue UI.
- Retrain the model monthly on newly labelled data using a scheduled job.

4.6 Mobile App for Reviewers & Supervisors

Claim reviewers are often away from their desks. A lightweight React Native (or Flutter) mobile app allowing makers and checkers to approve, reject, or return claims with a comment on their phone would compress turnaround times and reduce SLA breaches during high-volume periods.

Implementation:

- Scope to three screens: Queue list, Claim detail with PDF viewer, Approve/Reject/Return action with comment.
- Reuse existing REST API — no backend changes required.
- Add push notifications (Firebase Cloud Messaging) so officers are notified of new assignments without polling.

4.7 CI/CD Pipeline

The repository has no CI/CD configuration. Deployments are manual. A GitHub Actions pipeline should be the first infrastructure addition after tests exist.

Implementation:

- GitHub Actions: lint → type-check → test → build Docker image → push to registry on every PR merge to main.
- Separate staging and production environments with promotion gates.
- Add Dependabot for automated dependency security updates.
- Run OWASP ZAP or Semgrep in the pipeline for static application security testing (SAST) on every build.

5. Sector Context — InsurTech / East African Medical Insurance

ClaimsFlow operates in the intersection of **InsurTech** and **HealthTech** within the East African insurance market. Understanding the sector dynamics is essential for prioritising the right improvements.

Sector Factor	Current State	Implication for ClaimsFlow
---------------	---------------	----------------------------

Regulatory compliance	IRA (Kenya), FSRA (others)	SLA enforcement, mandatory audit logs, data residency
Fraud prevalence	10–15% of medical claims industry-wide	Invest in ML fraud scoring; rule-based detection alone is insufficient
Provider digitisation	Variable — large hospitals digitised, rural clinics paper-based	Keep batch PDF/scan upload as primary entry point; don't force EDI
Policyholder expectations	Shifting to real-time updates	SMS notifications already present; WhatsApp integration would expand reach
Market consolidation	M&A; activity in East Africa	Multi-tenant architecture would allow CIC to onboard subsidiary insurers
NHIF / SHA integration	National health fund interoperability	Future: HL7 FHIR API for cross-scheme claim data exchange
Mobile-first population	Kenya has 60%+ mobile internet penetration	Mobile reviewer app and provider WhatsApp submission are high-ROI investments

6. Prioritised Improvement Roadmap

The table below ranks all recommendations by effort (Low/Medium/High) and business impact. Tackle items in order — fixes that unblock everything else come first.

#	Area	Improvement	Effort	Impact
1	Testing	Write Jest unit + integration tests; set ≥80% coverage gate in CI	High	High
2	2FA Re-enable	Re-enable TOTP backend, make mandatory for privileged roles	Low	High
3	Security	Shorten JWT lifetime, tighten CORS, add rate limiting per endpoint	Low	High
4	CI/CD Pipeline	GitHub Actions: lint → test → build → deploy on merge to main	Medium	High
5	SLA Engine	BullMQ cron for SLA monitoring, auto-escalation, supervisor alerts	Medium	High
6	EDMS Integration	Implement authenticated document push on claim approval	Medium	High
7	eOxygen Payment	Implement payment initiation + reconciliation webhook	Medium	High
8	API Docs	Add @nestjs/swagger, decorate all controllers and DTOs	Low	Medium
9	DB Indexes	Add composite indexes; implement soft-deletes for compliance	Low	High
10	Observability	Centralised log aggregation + BullBoard queue dashboard	Medium	Medium
11	OCR Pre-Pop	Structured JSON from OCR → auto-fill claim form fields	High	High
12	Provider Portal	Read-only provider self-service claim status portal	Medium	Medium
13	Real-Time WS	WebSocket presence + live status updates on claim detail	Medium	Medium
14	ML Fraud Score	XGBoost fraud probability model as FastAPI microservice	High	High
15	Mobile App	React Native reviewer app: queue, detail, approve/reject	High	Medium

1	WhatsApp	WhatsApp-based claim status queries + document submission	Medium	Medium
6	Channel			

* Effort: Low < 1 week · Medium 1–3 weeks · High > 3 weeks

7. System Metrics Snapshot

Metric	Value	Status
Backend TypeScript files	93	✓
Frontend TypeScript / TSX files	80	✓
Total lines of code	~49,000	✓
NestJS modules	19	✓
REST API endpoints	179+	✓
Database models (Prisma)	30	✓
Frontend pages	27	✓
Background job processors	4	✓
Automated test files	0	■ Critical
Global exception filters	0	■ Gap
Swagger / OpenAPI spec	None	■ Gap
Active integrations (EDMS, eOxegen)	0 of 2	■ Pending
2FA backend controllers active	0	■ Disabled
Fraud detection rules	7	✓
OCR model providers supported	3	✓

8. Conclusion

ClaimsFlow is a commercially viable, architecturally sound insurance claims platform. The technology choices are appropriate, the domain modelling is correct, and the core workflows — batch ingestion, OCR, maker-checker approval, fraud flagging — function as designed. The codebase represents significant engineering investment and is closer to production than most platforms at this stage.

The path to a hardened production deployment is clear and achievable. The single most important action is establishing automated test coverage — everything else builds on that foundation. After that, completing the EDMS and eOxegen integrations closes the last gaps in the core claim lifecycle. Security hardening and SLA enforcement are compliance necessities that should be addressed before the platform handles live policyholder data.

The strategic additions — AI pre-population, ML fraud scoring, a provider self-service portal, and SLA enforcement — are not luxuries. In the competitive East African InsurTech market, they are the features that will differentiate CIC from legacy claims processors and enable ClaimsFlow to become a platform that providers, reviewers, and policyholders actively want to use.

Bottom line: Fix the foundation (tests, security, integrations) in the next sprint. Then build the differentiators (SLA engine, AI pre-population, provider portal) in the following quarter. The system will be ready for full production rollout within 2–3 months of focused engineering effort.